Teste Prático - Desenvolvedor Fullstack

Visão Geral

Crie uma aplicação **mobile (React Native + TypeScript)** e um **backend Node.js / TypeScript (opcional)** para gerenciar **Times** e **Tarefas**. O foco é na qualidade do código, clareza arquitetural, testes, UX mínima e documentação das decisões.

Entregáveis obrigatórios

- Repositório público com frontend e backend (pode ser monorepo).
- Backend em Node.js expondo uma API REST ou Graphql. Pode ser utilizado o framework de sua preferência (ex.: NestJS, Express, Fastify ou outro), desde que a arquitetura esteja bem organizada e documentada.
- Frontend em React Native + TypeScript consumindo a API.
- Formulários com validação (ex.: `react-hook-form` + `zod`).
- Gerenciamento de server state com **React Query**; state local (se necessário).
- Estilização sem `styled-components` (ex.: NativeWind / Dripsy / TailwindRN).
- README com instruções claras para executar o projeto.
- Seeds/fixtures (dados iniciais) para facilitar avaliação (ex.: 3 times e 10 tasks).

O que **não** está sendo prescrito

Não detalhamos **como** você deve implementar o banco de dados ou qual ORM usar:

- Propor a solução de persistência (ex.: Postgres, MongoDB, SQLite, etc.), justificar a escolha e descrever o modelo lógico (entidades e relacionamentos).

- Explicar como fará migrations/seeding e como irá garantir reprodutibilidade local.

Funcionalidades (o que precisa ser implementado)

Gestão de Times

- CRUD de times.
- Ao listar, retornar metadata (ex.: total/pagination).

Gestão de Tarefas

- CRUD de tarefas (Pendente | Em Progresso | Concluída).
- Uma tarefa pode pertencer a zero ou mais times (relacionamento).
- Endpoints de listagem com suporte a filtros: `teamId`, `status`, `search`, ordenação e paginação.

Frontend - telas mínimas

- Lista de times (tocar em um time filtra tarefas por time).
- Lista de tarefas (global e filtrada por time).
- Tela de criar/editar tarefa (form com validação).
- Visualizar tarefa (com ações rápidas: alterar status, deletar).
- Mostrar cor do time como label/chip na tarefa.

Figma Mockup: É um exemplo, porém pode ser feito com a sua própria ideia https://www.figma.com/design/4hOoWhreVcaQRXAa0ZVrMr/Avaliação-Minha-Claro-App?node-id=2802-395

API - contratos / payloads (exemplos)

Você pode manter livre o formato exato, mas documente no README.

Exemplos (sugestão):

- `GET /api/teams?limit=&offset=&search=`
- `POST /api/teams` `{ name, colorHex, description? }`
- `PUT /api/teams/:id`
- `DELETE /api/teams/:id`
- `GET /api/tasks?teamId=&status=&search=&limit=&offset=&sort=`
- `POST /api/tasks` `{ title, description?, status?, dueDate?, teamIds?: [] }`
- `PUT /api/tasks/:id`
- `DELETE /api/tasks/:id`

Resposta padrão sugerida: `200 { data, meta? }` e erros com um envelope `{ error: { code, message, details? } }`.

Requisitos de Aceitação

- CRUD de times e tarefas funcional via API e interface.
- Listagem de tarefas suporta filtros `teamId` e `status`, search e paginação.
- Botão/ação para alterar status (p.ex. marcar como Concluída) atualiza backend e UI.
- Cores dos times aparecem como label/chip nas tarefas.
- Formulários possuem validação (ex.: title mínimo 3 caracteres).
- Documentação com instruções de execução local e exemplos de requests.
- Seed/fixtures presentes para avaliação rápida.

Documentação Obrigatória

No **README** do seu projeto, inclua:

1. Decisões arquiteturais

Explique:

- Por que escolheu determinado banco

- Como modelou entidades e relacionamentos
- Como organiza camadas do backend
- Por que escolheu certos patterns (ex.: React Query, Zustand etc.)

2. Modelo de dados

Pode ser:

- Um diagrama simples; ou
- Uma explicação textual clara

3. Como rodar o projeto

Passo a passo para backend e frontend.

4. Exemplos de requisições

Ex.: cURL, Insomnia, Postman.

5. O que faria diferente em produção

Pontos como cache, logs, escalabilidade, segurança.

Diferenciais (opcionais)

- Offline-first (ex.: MMKV)
- Optimistic updates (ex.: React Query)
- Deploy backend em qualquer plataforma
- Testes mínimos: backend (unit + integration) e frontend (component test).
- Scripts: `start`, `test`, `seed`.

Entrega

- Submeter repositório público (GitHub ou GitLab)
- README completo e funcional
- Garantir que o projeto inicia sem erros a partir do passo-a-passo

Boa sorte!